ENCN205

Applied Data Analysis for Civil and Natural Systems

LECTURE 6

Vector Analysis

Slides and interactive demos (click or scan):
aardid.github.io/uc_205_gis_course

University of Canterbury | Te Whare Wānanga o Waitaha

Dr Alberto Ardid

Lecturer / Assistant Professor
Department of Civil and Environmental Engineering
University of Canterbury, New Zealand

E319 |
alberto.ardid@canterbury.ac.nz

Spatial analysis = answering questions by combining datasets.

A single dataset tells you what exists.

Combining datasets tells you what matters.

Today's toolkit:

Attribute queries and table joins

Spatial joins (gpd.sjoin)

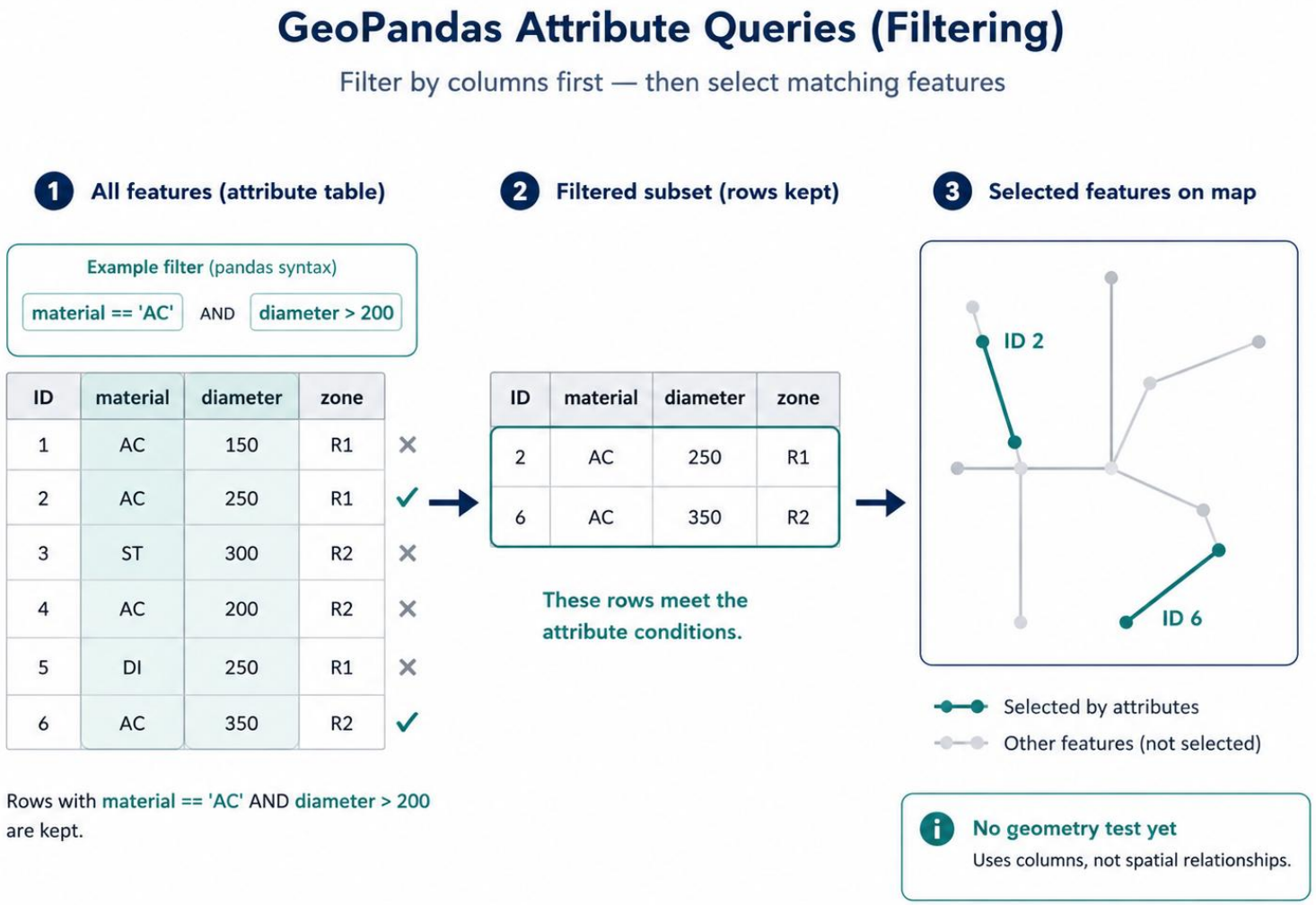
Overlay operations (gpd.overlay)

Buffers, dissolve, clipping

Attribute Queries

Filter GeoDataFrames using standard pandas syntax:

```
gdf[gdf['material'] == 'AC']              # exact match
gdf[gdf['diameter'] > 200]                # numeric comparison
gdf[gdf['zone'].isin(['R1', 'R2'])]       # membership
gdf.query("age > 50 and material == 'AC'")    # query string
```



Key point: filtering by attributes is NOT spatial analysis - it doesn't use geometry.

But it's almost always the first step: reduce your data before spatial operations.

Table Joins — .merge()

Links records by a shared column (key):

```
gdf.merge(df, on='pipe_id')      # shared key
gdf.merge(df, left_on='zone_code',
          right_on='code')    # different names
```

Join types:

inner — only matching rows (default)

left — keep all rows from the GeoDataFrame

outer — keep all rows from both

1. Spatial data (pipes.shp)

pipe_id	material	diameter	geometry
P001	AC	225	
P002	PVC	150	
P003	AC	300	
P004	PVC	200	

GeoDataFrame (has geometry)

+

2. Attribute table (ratings.csv)

pipe_id	rating	date
P001	Good	2024-01-10
P002	Fair	2024-01-12
P003	Poor	2024-01-15
P005	Good	2024-01-18

Regular DataFrame (no geometry)

3. Result after merge (inner join)

pipe_id	material	diameter	geometry	rating	date
P001	AC	225		Good	2024-01-10
P002	PVC	150		Fair	2024-01-12
P003	AC	300		Poor	2024-01-15

Spatial data enriched with ratings

Common use case:

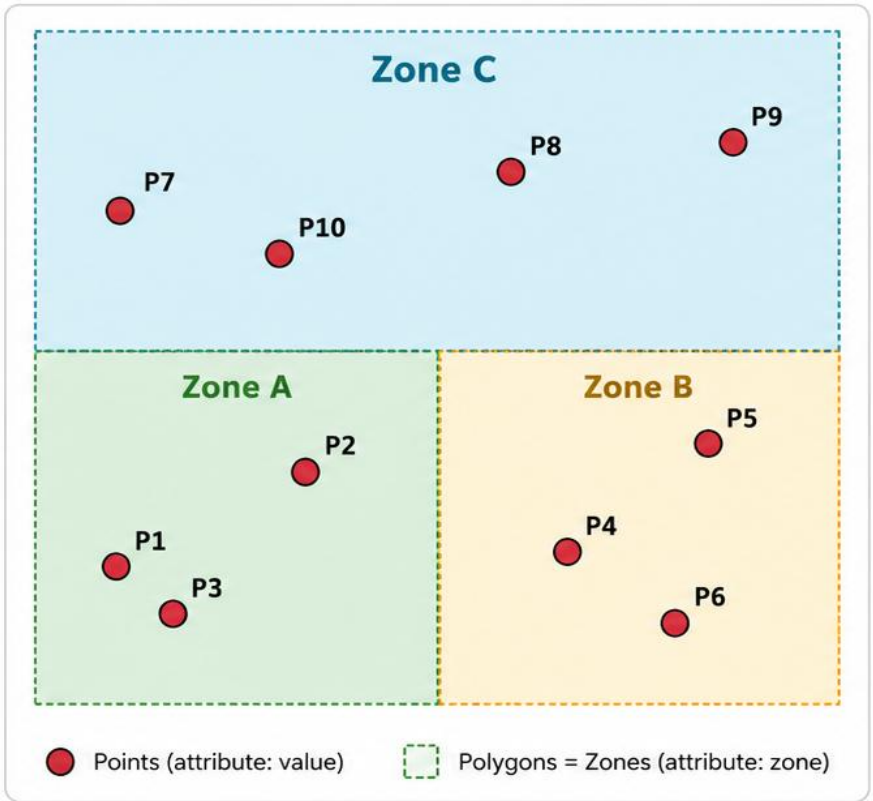
You have a shapefile of pipes (geometry + pipe_id) and a CSV of condition ratings (pipe_id + rating + date).

.merge() links the rating data to the spatial data so you can map pipe condition.

No geometry involved - this is a standard relational join.

Spatial Join — gpd.sjoin(points, polygons, predicate='within')

1 Input: Points + Polygons (Zones)



2 Spatial Join

Match each point to the polygon (zone) it falls within.

```
gpd.sjoin(
   points,
   polygons,
   predicate='within'
)
```

3 Output: Points with Zone Attribute

point_id	value	zone
P1	12	Zone A
P2	7	Zone A
P3	19	Zone A
P4	5	Zone B
P5	23	Zone B
P6	8	Zone B
P7	15	Zone C
P8	11	Zone C
P9	3	Zone C
P10	20	Zone C

What happened? Each point was assigned the zone (polygon) it falls within using the 'within' predicate. Points on boundaries are excluded by 'within' (not counted in any zone).

Links records by geographic relationship, not by a shared column.

Predicates: 'within', 'contains', 'intersects' - choose based on the question.

Spatial Join vs Table Join

Table Join (.merge)

Links by shared column (key)

Both tables must have matching IDs

No geometry needed

Use when: data comes from the same system

Example: pipe shapefile + pipe condition CSV

Spatial Join (gpd.sjoin)

Links by geographic relationship

No shared column needed

Both must be GeoDataFrames

Use when: data comes from different sources

Example: buildings + flood zones (different agencies)

Rule of thumb: if the datasets share an ID column, use .merge(). If not, use gpd.sjoin().

The One-to-Many Problem

Warning: gpd.sjoin() can return MORE rows than your input!

This happens when one feature matches multiple features in the other layer:

A building that sits on the boundary of two zones -> 2 rows

A long pipe that crosses three soil polygons -> 3 rows

Overlapping polygons (e.g. flood zones + hazard zones) -> multiple matches

Always check: len(result) vs len(input)

If result is larger, you have duplicates — decide how to handle them:

Keep all (if you want every relationship)

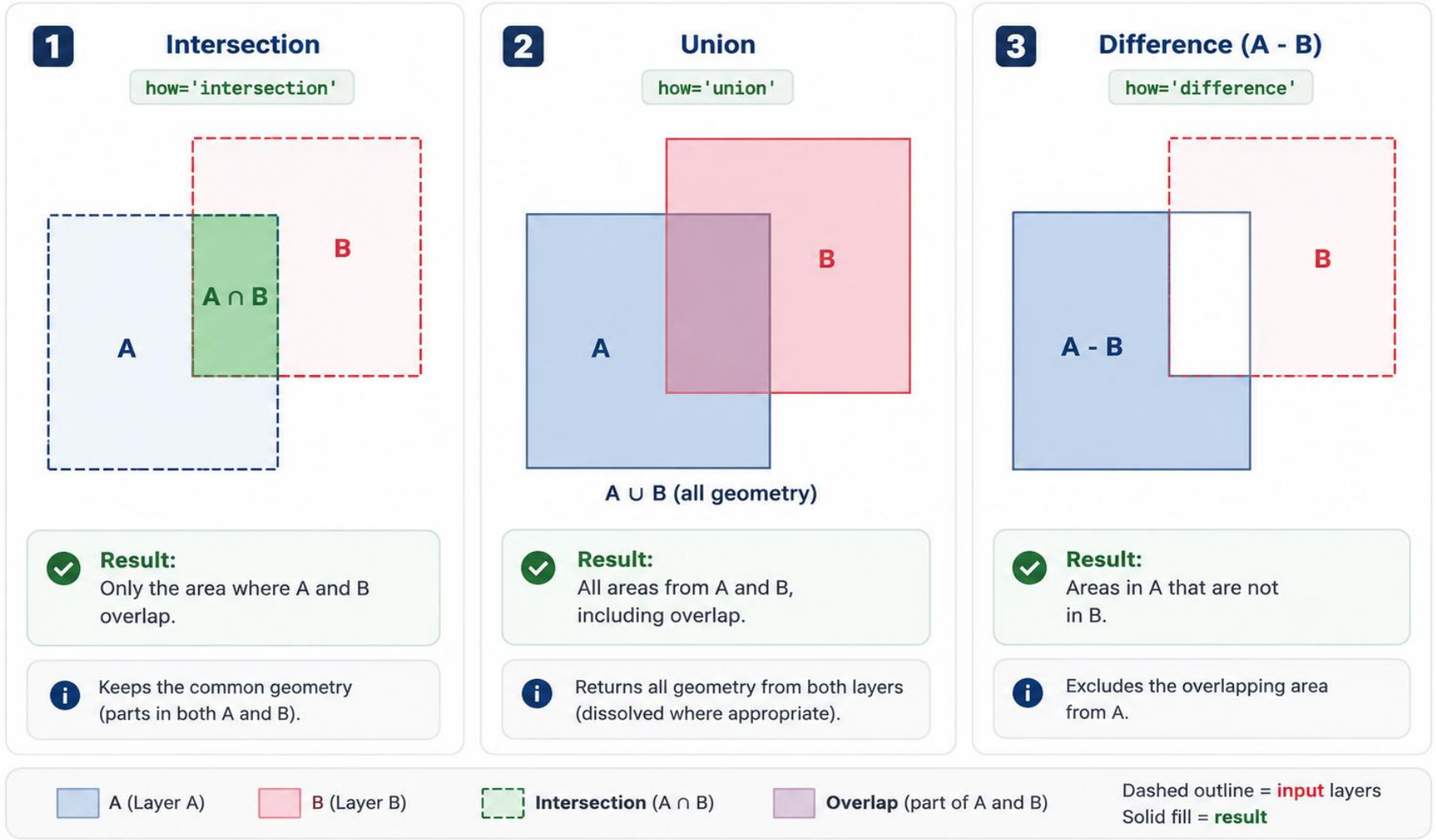
Drop duplicates (keep first match)

Dissolve/aggregate (sum, mean, count)

Always ask: 'did my row count change - and why?'

Overlay Operations — gpd.overlay(A, B, how=...)

Overlay combines two vector layers (A and B) to produce a new layer based on spatial relationships.



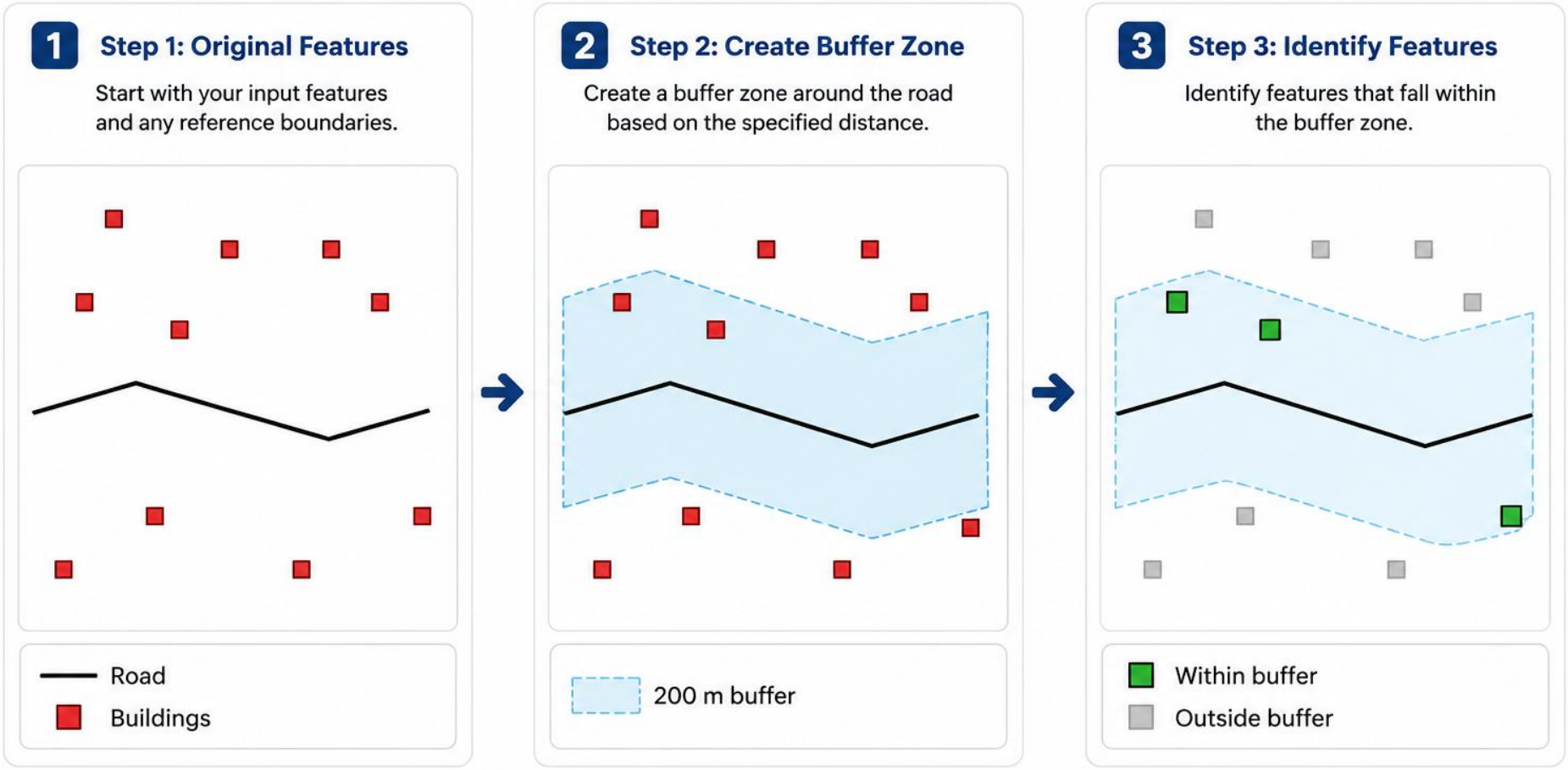

```
gpd.overlay(gdf1, gdf2, how='intersection')   # also 'union', 'difference', 'symmetric_difference'
```

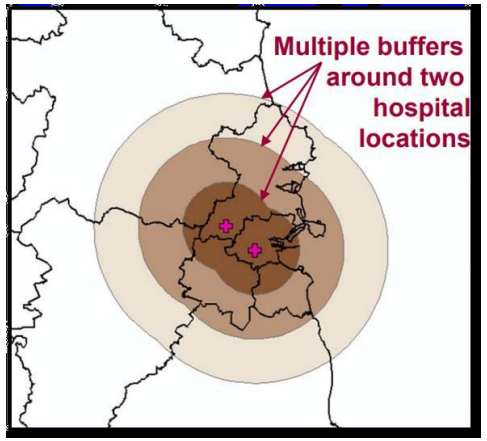
Always ask: 'which layer's attributes survive?'

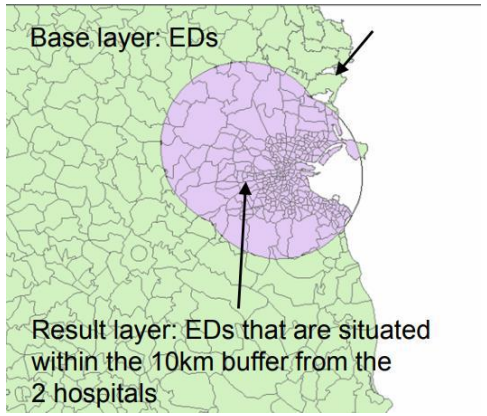
Unlike sjoin, overlay creates NEW geometries by cutting polygons at boundaries.

Buffer Analysis Workflow

Create buffer zones around features to identify nearby features within a specified distance.







What this does: Buffers help you find features (e.g., buildings) that are within a certain distance of another feature (e.g., a road) for proximity-based analysis.

```
gdf['geometry'] = gdf.geometry.buffer(distance)   # distance in CRS units (metres for NZTM)
```

Creates proximity zones -- essential for 'within X metres of...' questions.

Workflow: Buffer + Spatial Join

"Which buildings are within 100m of the coast?"

Step 1: Load coastline and buildings (check CRS = NZTM)

Step 2: coast_buffer = coastline.buffer(100) # 100m zone

Step 3: exposed = gpd.sjoin(buildings, coast_buffer, predicate='within')

Step 4: print(f'{len(exposed)} buildings within 100m of coast')

This is the most common spatial analysis pattern in engineering:

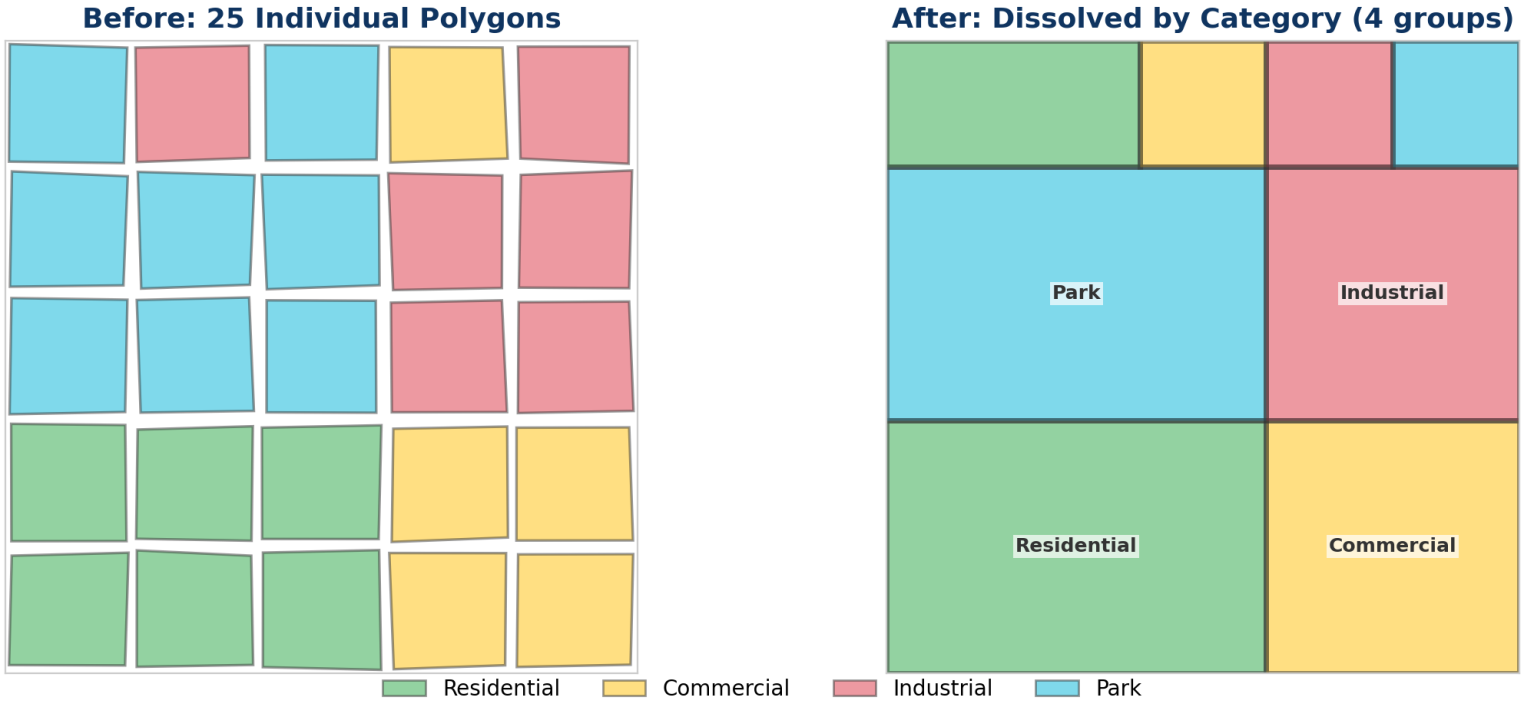
Buffer a feature of interest -> Spatial join to find what's nearby

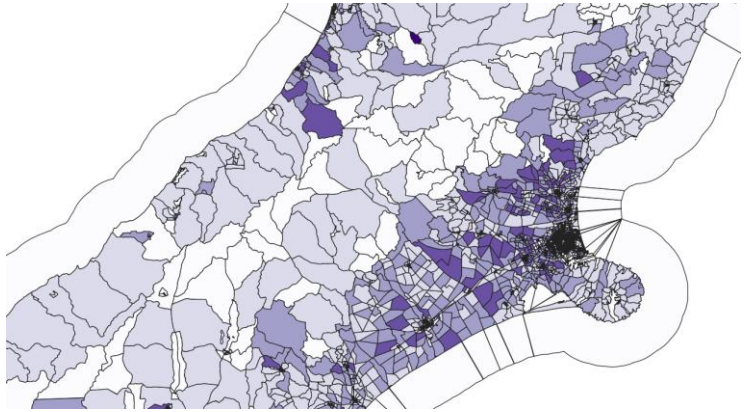
Variations: within 50m of a fault line? Within 500m of a school? Within 1km of a landfill?

Always ask: 'are my units metres?'

Dissolve and Spatial Aggregation

Dissolve — gdf.dissolve(by='land_use', aggfunc='sum')





gdf.dissolve(by='land_use', aggfunc='sum') — merges geometry + aggregates attributes

Like SQL GROUP BY but for spatial data — combines geometry and computes statistics.

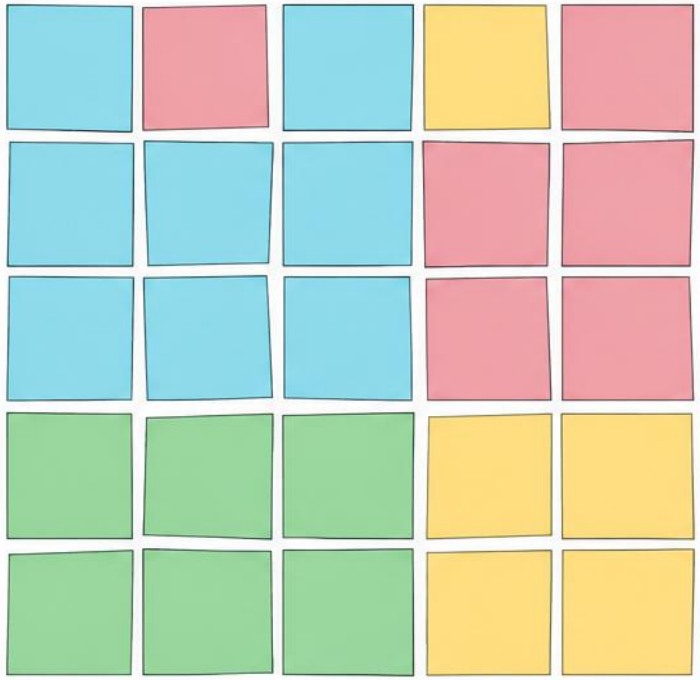
Dissolve and Spatial Aggregation

Dissolve — gdf.dissolve(by='land_use', aggfunc='sum')

Combine polygons that share the same category (land_use) into single geometries and sum their attributes.

1 Before: 25 Individual Polygons

Each polygon represents a parcel with a land use category.



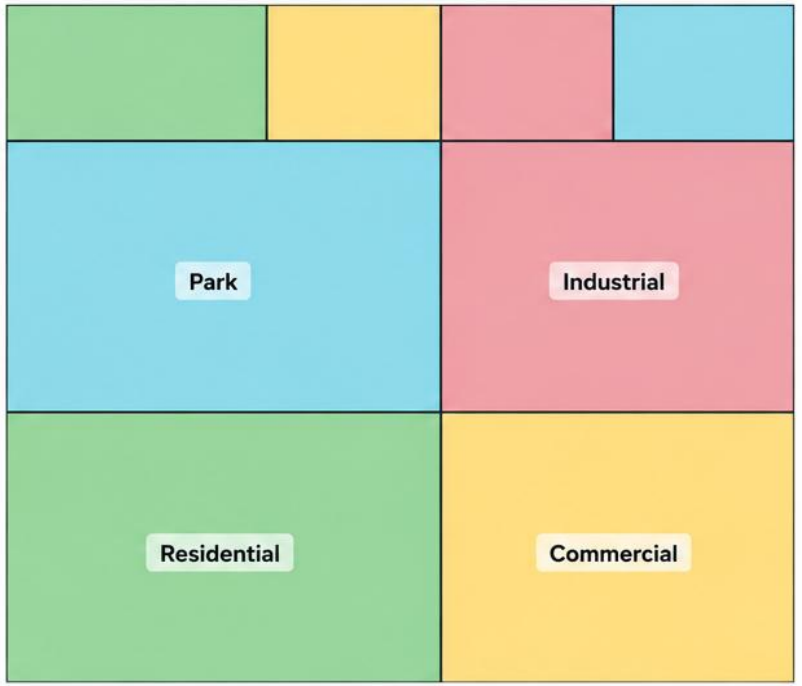
Residential Commercial Industrial Park

gdf.dissolve(
by='land_use',
aggfunc='sum',
)

Polygons with the same land_use are merged and their numeric attributes are summed.

2 After: Dissolved by Category (4 groups)

All polygons with the same land use are dissolved into single geometries.



Residential Commercial Industrial Park

What happened? Dissolve groups features by the specified field (land_use) and merges all geometries in each group into a single geometry. Numeric fields are aggregated using the specified function (sum in this case).

```
gdf.dissolve(
   by='land_use',
   aggfunc='sum'
)
```

LIVE DEMO

L06_VectorAnalysis.html

Buffer a coastline, spatial join buildings, calculate exposure.
Interactive — adjust the buffer distance and see results update.

Scan to open the demo on your phone

aardid.github.io/uc_205_gis_course/L06_VectorAnalysis.html

All slides & demos: aardid.github.io/uc_205_gis_course

Clipping - Extracting Features Within a Boundary

gpd.clip(gdf, boundary) — cuts features to a study area

What it does:

Points: keeps only those inside the boundary

Lines: cuts lines at the boundary edge, keeps interior segments

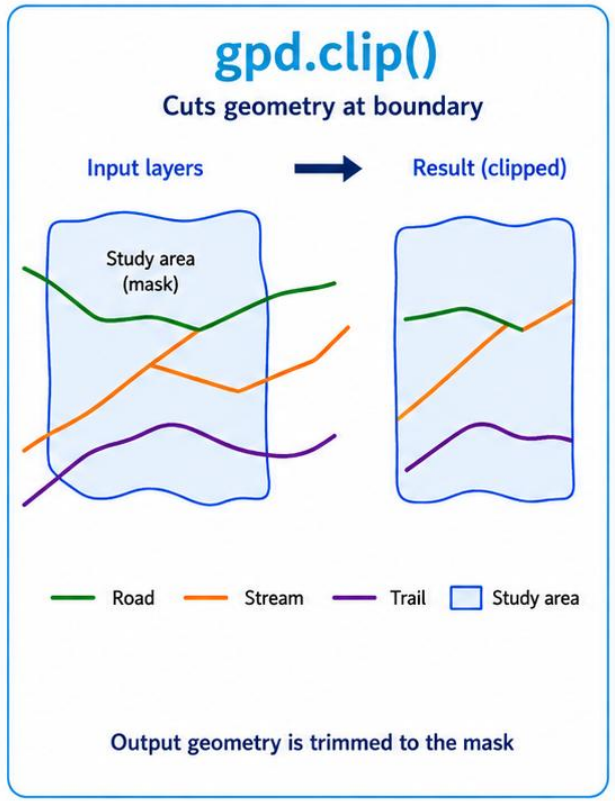
Polygons: cuts polygons at the boundary, keeps interior portions

Common scenario:

You have national-scale pipe data but only need Christchurch.

clipped = gpd.clip(pipes, chch_boundary)

Clipping MODIFIES geometry (cuts it). Spatial query only SELECTS features.



Always ask: 'do I need to cut, or just select?'

Clip vs Spatial Query vs Overlay

gpd.clip()

Cuts geometry at boundary

Result fits exactly inside

Use for: study area extraction

gpd.sjoin()

Selects features by location

Geometry unchanged

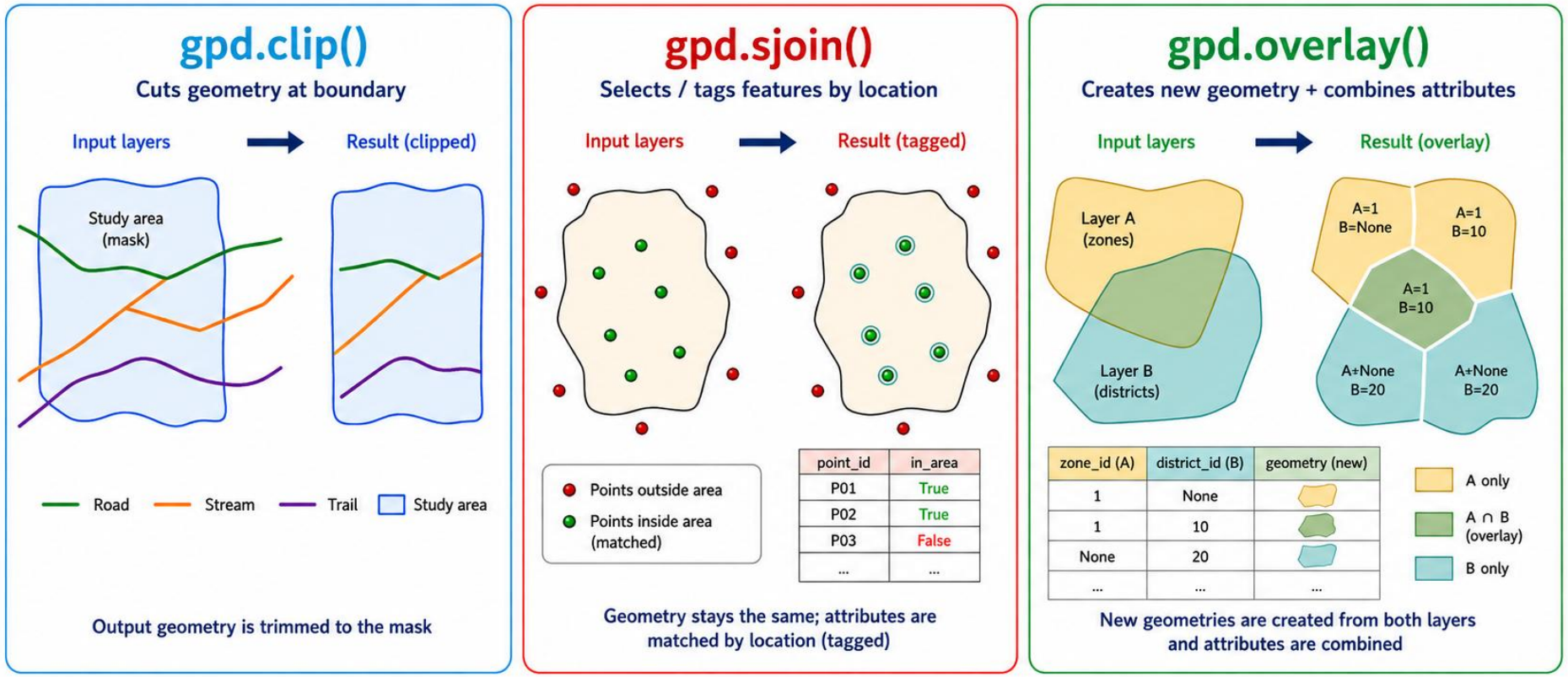
Use for: finding what's near/inside

gpd.overlay()

Creates new cut geometry

Combines attributes from both

Use for: polygon-on-polygon analysis



Choose based on the question: Do you need to CUT geometry? TAG features? Or COMBINE layers?
Wrong tool = wrong result or wasted computation.

KEY CONCEPTS

Today's Takeaways

1. Table join (.merge) links by shared column; spatial join (sjoin) links by location
2. sjoin TAGS features; overlay CUTS geometry — choose based on the question
3. Buffer + sjoin is the core pattern for proximity analysis
4. Dissolve aggregates features spatially — like groupby for geometry
5. Always: check CRS match, check row counts after sjoin, filter before spatial ops

NEXT LECTURE

Lecture 7: Cartography Essentials

Common Pitfalls

1. CRS mismatch

Both layers MUST share the same CRS. If not: gdf = gdf.to_crs(other.crs)

Buffer in WGS84 (degrees) gives nonsensical results — always use NZTM (metres).

2. One-to-many duplicates

sjoin can create more rows than input. Always check len(result) vs len(input).

3. Forgetting to filter first

Spatial operations are slow on large datasets. Filter attributes first, then do spatial ops.

4. Choosing the wrong operation

sjoin when you need overlay (or vice versa) gives wrong results, not just wrong speed.

Discussion

Design a workflow to find all schools within 1 km of a tsunami zone.

Think about:

What datasets do you need?

What CRS should you use?

What operations, in what order?

How would you present the results to a council planner?

Hint: it's the same buffer -> sjoin pattern you just learned.

Workflow 1: Coastal Exposure Assessment

Question: How many buildings per suburb are within 200m of the coast?

Step 1: coast_buffer = coastline.buffer(200)

Create a 200m proximity zone around the coastline

Step 2: exposed = gpd.sjoin(buildings, coast_buffer, predicate='within')

Tag each building that falls within the buffer zone

Step 3: summary = exposed.dissolve(by='suburb', aggfunc='count')

Aggregate: count exposed buildings per suburb

Step 4: summary.plot(column='building_id', legend=True)

Map the result — a choropleth of exposure by suburb

3 operations: buffer -> sjoin -> dissolve. This pattern solves many engineering problems.

Workflow 2: Pipes in Poorly-Drained Soil

Question: What length of AC pipe runs through poorly-drained soil?

Step 1: ac_pipes = pipes[pipes['material'] == 'AC']

Filter to asbestos cement pipes only (attribute query)

Step 2: pipe_soil = gpd.overlay(ac_pipes, soil_zones, how='intersection')

Split pipes at soil zone boundaries — each segment has a soil type

Step 3: poor = pipe_soil[pipe_soil['drainage'] == 'poor']

Filter to poorly-drained zones

Step 4: total_km = poor.geometry.length.sum() / 1000

Calculate total pipe length in km

Key: overlay (not sjoin) because we need to CUT pipe geometry at soil boundaries.